3.3. Odometry-based motion model

Odometry provides an estimate of the robot motion from its proprioceptive sensors, typically the **wheel encoders** in a differential-drive robot. In practice, the encoders measure how much each wheel has rotated over a short time interval, and this information can be converted into an incremental displacement of the robot.

As a curiosity, the term *odometry* comes from the Greek words ὁδός (*odos*, route) and μέτρον (*metron*, measurement), literally meaning *measurement of the route*.



Fig. 1: Example of a wheel encoder used to sum pulses and compute the robot pose.

From encoder pulses to wheel rotation

An incremental encoder generates a number of pulses as the wheel rotates. If the encoder provides n_{total} pulses per complete revolution, the angular increment associated with a single pulse is

$$\alpha = \frac{2\pi}{n_{\text{total}}} \qquad (\text{radians}).$$

Therefore, if n_R and n_L pulses are detected for the right and left wheels during a time interval Δt, their corresponding angular increments are

$$\Delta\varphi_R = n_R\alpha, \qquad \Delta\varphi_L = n_L\alpha.$$

These angular increments can be converted into the distances traveled by each wheel:

$$\Delta s_R = r_R \Delta \varphi_R, \qquad \Delta s_L = r_L \Delta \varphi_L,$$

where r_R and r_L are the radii of the right and left wheels, respectively.

From wheel motion to robot motion

Assuming a differential-drive robot and constant curvature during the short motion interval, the displacement of the midpoint of the wheel axis is characterized by two quantities:

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2},$$

which represents the distance traveled along the arc by the midpoint of the wheel axis, and

$$\Delta \theta = \frac{\Delta s_R - \Delta s_L}{l},$$

which represents the change in robot orientation, with l being the distance between the wheels.

For $\Delta \theta \neq 0$, the robot follows an arc of radius

$$R = \frac{\Delta s}{\Delta \theta}.$$

The corresponding pose increment, expressed in the robot local reference frame, is then

$$\Delta \mathbf{x}_t^{\text{odo}} = \begin{bmatrix} \Delta x_t \\ \Delta y_t \\ \Delta \theta_t \end{bmatrix} = \begin{bmatrix} \frac{\Delta s}{\Delta \theta} \sin(\Delta \theta) \\ \frac{\Delta s}{\Delta \theta} (1 - \cos(\Delta \theta)) \\ \Delta \theta \end{bmatrix}.$$

If $\Delta \theta = 0$, both wheels have traveled the same distance and the robot moves along a straight line:

$$\Delta \mathbf{x}_t^{\text{odo}} = \begin{bmatrix} \Delta s \\ 0 \\ 0 \end{bmatrix}.$$

Thus, the complete process can be summarized as

$$(\Delta \varphi_R, \Delta \varphi_L) \rightarrow (\Delta s_R, \Delta s_L) \rightarrow (\Delta s, \Delta \theta) \rightarrow \Delta \mathbf{x}_t^{\text{odo}}.$$

Accumulating odometry

The firmware of the robotic base typically performs these computations at a **high rate** (e.g. **100 Hz**), generating a sequence of small pose increments. These increments are accumulated using the **composition of poses**, a tool already introduced in previous sections:

$$\mathbf{x}_t^{\text{odo}} = \mathbf{x}_{t-1}^{\text{odo}} \oplus \Delta\mathbf{x}_t^{\text{odo}}.$$

The resulting odometry pose can then be made available to the rest of the robotic system at a lower rate (e.g. **10 Hz**).

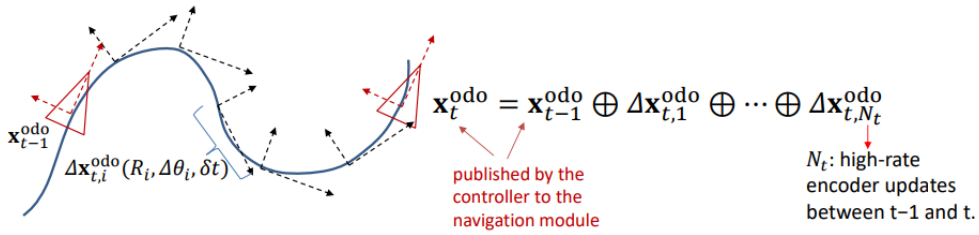


Fig. 2: Example of composition of poses based on odometry.

Therefore, between two odometry poses reported by the robotic base, several smaller pose increments may have been internally computed and accumulated by the firmware. For example, if the internal computation runs at 100 Hz while odometry is reported at 10 Hz, approximately 10 internal pose increments are accumulated between two consecutive reported odometry poses.

The motion model

The **odometry motion model** uses the pose increment obtained from two consecutive odometry readings as the input to the motion model. Although odometry is technically a measurement rather than a control command, it is treated as the model input for convenience.

The odometry increment between times $t-1$ and t is defined as

$$\Delta\mathbf{x}_t^{\text{odo}} = (\ominus\mathbf{x}_{t-1}^{\text{odo}}) \oplus \mathbf{x}_t^{\text{odo}} = \begin{bmatrix} \Delta x_t \\ \Delta y_t \\ \Delta\theta_t \end{bmatrix}.$$

Therefore, the odometry motion model can be expressed as

$$\mathbf{x}_t = g_o\left(\mathbf{x}_{t-1}, \Delta\mathbf{x}_t^{\text{odo}}\right),$$

where $\mathbf{x}_{t-1}$ is the previous robot pose and $\mathbf{u}_t = \Delta\mathbf{x}_t^{\text{odo}}$ is the pose increment measured by odometry.

In this work we are going to play with the **probabilistic** variant of this motion model using two different forms:

- **Analytical form**, where the odometry input is directly represented as a pose increment:

$$\Delta \mathbf{x}_t^{\text{odo}} = \begin{bmatrix} \Delta x_t \\ \Delta y_t \\ \Delta \theta_t \end{bmatrix}.$$

- **Sample-based form**, where the same motion is decomposed into an initial rotation, a straight-line displacement, and a final rotation:

$$\mathbf{u}_t^{\text{odo}} = \begin{bmatrix} \bar{\theta}_1 \\ \bar{d} \\ \bar{\theta}_2 \end{bmatrix}.$$

> ℹ️ **Usage:** The ***velocity motion model*** is mainly useful when the robot motion is described through commanded linear and angular velocities, as is common in motion planning. In contrast, the ***odometry motion model*** uses the motion actually measured by the robot sensors and is therefore particularly suitable for keeping track of the robot pose. However, odometry information is only available after the corresponding motion has been measured.

In [1]:
```
# In case of using Google Colab, mount Google Drive and enable interactive plots
try:
    from google.colab import drive, output
    drive.mount('/content/drive')
    # Change directory to this chapter's folder in your Drive copy of the course repo:
    %cd '/content/drive/MyDrive/Colab Notebooks/uma_robotics_2027/Chapter 03. Robot motion'
    %pip install -q ipympl
    # Make matplotlib detect the freshly installed ipympl backend:
    from matplotlib.backends.registry import backend_registry
    backend_registry.__init__()
    output.enable_custom_widget_manager()
except ImportError:
    pass

%matplotlib inline

# IMPORTS
import numpy as np
from numpy import random
import matplotlib.pyplot as plt
from scipy import stats
from IPython.display import display, clear_output
```

```
import time

import sys
sys.path.append("..")
sys.path.append("../..")
from utils.DrawRobot import DrawRobot
from utils.PlotEllipse import PlotEllipse
from utils.pause import pause
from utils.Jacobians import J1, J2
from utils.tcomp import tcomp
```

OPTIONAL

Let's reproduce the odometry computation performed by the robot base firmware!

1. Implement a function that, given the encoder pulses detected in both wheels, computes their angular increments $\Delta\varphi_R$ and $\Delta\varphi_L$, and the corresponding wheel displacements Δs_R and Δs_L.

2. Implement a second function that computes the arc length Δs and heading change $\Delta\theta$, and uses them to obtain the odometry pose increment $\Delta\mathbf{x}_t^{\text{odo}} = [\Delta x_t, \Delta y_t, \Delta\theta_t]^T$. Remember to handle the straight-line case ($\Delta\theta = 0$).

3. Finally, given two sequences of encoder pulse counts (one for each wheel) and an initial robot pose, use the previous functions to compute the odometry pose increment for each pair of pulse counts. Starting from the initial pose, **update the robot pose after each increment using pose composition**. Return the complete sequence of estimated poses and the final odometry pose.

Odometría a partir de los pulsos de los encoders

Se implementan las tres funciones que pide el enunciado, siguiendo la cadena $(n_R, n_L) \to (\Delta\varphi_R, \Delta\varphi_L) \to (\Delta s_R, \Delta s_L) \to (\Delta s, \Delta\theta) \to \Delta\mathbf{x}_t^{\text{odo}}$:

1. `wheel_increments()`: $\alpha = 2\pi / n_{\text{total}}$, $\Delta\varphi = n\,\alpha$ y $\Delta s = r\,\Delta\varphi$ para cada rueda.
2. `pose_increment()`: $\Delta s = (\Delta s_R + \Delta s_L)/2$, $\Delta\theta = (\Delta s_R - \Delta s_L)/l$ y el incremento local del arco, con el caso recto $\Delta\theta = 0 \Rightarrow \Delta\mathbf{x} = [\Delta s, 0, 0]^T$. Se usa una tolerancia en lugar de `== 0`, porque con radios distintos $\Delta\theta$ puede ser un número minúsculo pero no nulo; cuando $|\Delta\theta| < 10^{-9}$ rad las dos fórmulas difieren en menos de un nanómetro.
3. `odometry()`: recorre las dos secuencias de pulsos y acumula los incrementos con la composición de poses, $\mathbf{x}_t^{\text{odo}} = \mathbf{x}_{t-1}^{\text{odo}} \oplus \Delta\mathbf{x}_t^{\text{odo}}$ (`tcomp`). Devuelve todas las poses y la pose final.

Primero se comprueban tres casos con solución conocida: avance recto, giro sobre sí mismo y la propiedad de que, con curvatura constante, componer muchos arcos pequeños da exactamente el mismo resultado que un único arco con todos los pulsos.

In [2]:

```python
def wheel_increments(n_R, n_L, n_total, r_R, r_L):
    '''Encoder pulses -> angular increments (rad) and travelled distances
(m) of the right and left wheels.'''
    alpha = 2 * np.pi / n_total
    dphi_R, dphi_L = n_R * alpha, n_L * alpha
    return dphi_R, dphi_L, r_R * dphi_R, r_L * dphi_L

def pose_increment(ds_R, ds_L, l, tol=1e-9):
    '''Wheel distances -> arc length, heading change and local pose
increment [dx, dy, dtheta]^T.'''
    ds = (ds_R + ds_L) / 2
    dtheta = (ds_R - ds_L) / l
    if abs(dtheta) < tol:  # straight line
        return ds, dtheta, np.vstack([ds, 0., 0.])
    R = ds / dtheta
    return ds, dtheta, np.vstack([R * np.sin(dtheta), R * (1 -
np.cos(dtheta)), dtheta])

def odometry(pulses_R, pulses_L, x0, n_total, r_R, r_L, l):
    '''Accumulates the odometry increments of each pair of pulse counts by
pose composition.'''
    poses = [x0]
    for n_R, n_L in zip(pulses_R, pulses_L):
        *_, ds_R, ds_L = wheel_increments(n_R, n_L, n_total, r_R, r_L)
        poses.append(tcomp(poses[-1], pose_increment(ds_R, ds_L, l)[2]))
    return np.hstack(poses), poses[-1]

# Robot base parameters: 1024 pulses per revolution, 5 cm wheels, 30 cm
between wheels
n_total, r_R, r_L, l = 1024, 0.05, 0.05, 0.30
step = 2 * np.pi * r_R / n_total  # distance per pulse
origin = np.vstack([0., 0., 0.])

# 1) Straight line: same pulses in both wheels
_, straight = odometry([100] * 10, [100] * 10, origin, n_total, r_R, r_L,
l)
assert np.allclose(straight, [[1000 * step], [0], [0]])
# 2) Rotation in place: opposite pulses -> ds = 0, only the heading changes
_, spin = odometry([50], [-50], origin, n_total, r_R, r_L, l)
assert np.allclose(spin, [[0], [0], [100 * step / l]])
# 3) Constant curvature: 10 small arcs composed == a single arc with all
the pulses
_, small_arcs = odometry([30] * 10, [20] * 10, origin, n_total, r_R, r_L,
l)
_, single_arc = odometry([300], [200], origin, n_total, r_R, r_L, l)
assert np.allclose(small_arcs, single_arc)
print(f"Distance per pulse: {1000 * step:.3f} mm")
print(f"Straight (1000 pulses each wheel):  {straight.ravel().round(4)}")
print(f"Spin (+50 / -50 pulses):            {spin.ravel().round(4)}")
```

```
print(f"10 arcs (30/20) vs 1 arc (300/200): {small_arcs.ravel().round(4)}
== {single_arc.ravel().round(4)}")
```

```
Distance per pulse: 0.307 mm
Straight (1000 pulses each wheel):  [0.3068 0.     0.    ]
Spin (+50 / −50 pulses):            [0.     0.     0.1023]
10 arcs (30/20) vs 1 arc (300/200): [0.0766 0.0039 0.1023] == [0.0766 0.0039
0.1023]
```

Ahora se reproduce el funcionamiento del firmware con un recorrido completo. Se simula una base que se mueve durante 15 s con cuatro tramos de velocidad constante (v, ω): recto, media vuelta a la izquierda, giro de 90° sobre sí mismo y recto otra vez. Con ellos:

- Las velocidades de las ruedas son $v_{R,L} = v \pm \omega l/2$. Los pulsos de cada periodo de 10 ms (**100 Hz**) se obtienen como un encoder real: contando cuántas marcas se han cruzado, $n_k = \lfloor \varphi(t_k)/\alpha \rfloor - \lfloor \varphi(t_{k-1})/\alpha \rfloor$. Así la cuantización no acumula sesgo, porque el error de cada lectura nunca supera un pulso.
- `odometry()` compone los 1500 incrementos. La pose solo se «publica» cada 10 incrementos (**10 Hz**), y entre dos poses publicadas se calcula el incremento que usa el modelo de movimiento, $\Delta\mathbf{x}_t^{\text{odo}} = (\ominus\mathbf{x}_{t-1}^{\text{odo}}) \oplus \mathbf{x}_t^{\text{odo}}$, con `inv_tcomp`.
- Como referencia se usa la trayectoria ideal: la misma composición, pero con los ángulos exactos de las ruedas, sin cuantizar.

In [3]:
```
from utils.tcomp import inv_tcomp

# Velocity profile (v [m/s], w [rad/s], duration [s]) and firmware / report
rates
segments = [(0.4, 0., 3.), (0.3, 0.5, np.pi / 0.5), (0., np.pi / 4, 2.),
(0.4, 0., 15. - 5. - np.pi / 0.5)]
f_firmware, f_report = 100, 10
dt = 1 / f_firmware

# Continuous wheel angles phi(t) at each firmware tick
v_wheels = []
for v, w, duration in segments:
    n_ticks = int(round(duration * f_firmware))
    v_wheels += [(v + w * l / 2, v - w * l / 2)] * n_ticks
v_wheels = np.array(v_wheels)
phi = np.vstack([np.zeros((1, 2)), np.cumsum(v_wheels * dt / np.array([r_R,
r_L]), axis=0)])

# Encoder: number of marks crossed in each period (no accumulated
quantization bias)
alpha = 2 * np.pi / n_total
pulses = np.diff(np.floor(phi / alpha), axis=0).astype(int)

x0 = np.vstack([0., 0., np.pi / 2])
poses, final_pose = odometry(pulses[:, 0], pulses[:, 1], x0, n_total, r_R,
r_L, l)
ideal, ideal_final = odometry(*(np.diff(phi, axis=0) / alpha).T, x0,
n_total, r_R, r_L, l)
```

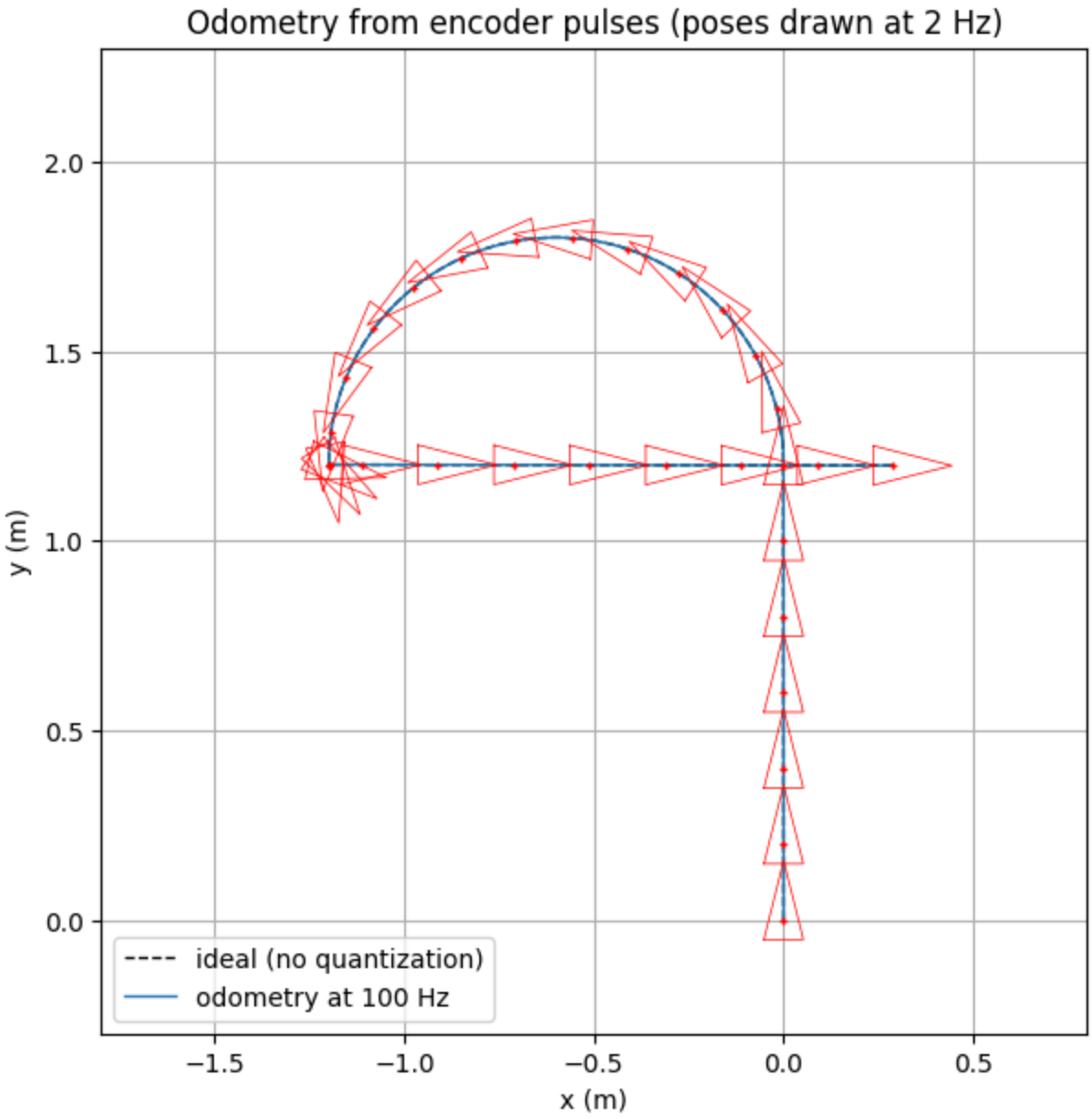
```
reported = poses[:, ::f_firmware // f_report]
increments = np.hstack([inv_tcomp(reported[:, [k - 1]], reported[:, [k]])
for k in range(1, reported.shape[1])])

print(f"{len(pulses)} firmware increments at {f_firmware} Hz ->
{reported.shape[1] - 1} reported odometry poses at {f_report} Hz")
print(f"Final odometry pose:  {final_pose.ravel().round(4)}")
print(f"Ideal final pose:     {ideal_final.ravel().round(4)}")
print(f"Quantization error:   {1000 * np.linalg.norm(final_pose[:2] -
ideal_final[:2]):.2f} mm, "
      f"{np.degrees(abs(final_pose[2, 0] - ideal_final[2, 0])):.3f} deg")
print("\nSome odometry increments between reported poses, (-)x_{t-1} (+)
x_t  [dx, dy, dtheta]:")
for k in (0, 50, 120, 140):
    print(f"  t = {k / f_report:4.1f} -> {(k + 1) / f_report:4.1f} s:
{increments[:, k].round(4)}")

fig, ax = plt.subplots(figsize=(7, 7))
ax.plot(ideal[0], ideal[1], 'k--', linewidth=1, label='ideal (no
quantization)')
ax.plot(poses[0], poses[1], '-', color='tab:blue', linewidth=1,
label=f'odometry at {f_firmware} Hz')
ax.set(aspect='equal', xlim=(-1.8, 0.8), ylim=(-0.3, 2.3), xlabel='x (m)',
ylabel='y (m)',
       title=f'Odometry from encoder pulses (poses drawn at {f_report // 5}
Hz)')
plt.sca(ax)
for k in range(0, reported.shape[1], 5):
    DrawRobot(fig, ax, reported[:, [k]])
ax.grid(True)
ax.legend(loc='lower left')
plt.show()
```

```
1500 firmware increments at 100 Hz -> 150 reported odometry poses at 10 Hz
Final odometry pose:  [ 0.2881  1.1984 -0.002 ]
Ideal final pose:     [ 0.288   1.1986 -0.0016]
Quantization error:   0.22 mm, 0.026 deg

Some odometry increments between reported poses, (-)x_{t-1} (+) x_t  [dx,
dy, dtheta]:
  t =  0.0 ->  0.1 s: [0.0399 0.     0.    ]
  t =  5.0 ->  5.1 s: [0.0299 0.0007 0.0501]
  t = 12.0 -> 12.1 s: [ 0.04  -0.    -0.001]
  t = 14.0 -> 14.1 s: [0.04  0.    0.001]
```



Observaciones:

- **Las tres comprobaciones se cumplen.** Con pulsos iguales el robot avanza en línea recta (1000 pulsos × 0.307 mm/pulso = 0.307 m). Con pulsos opuestos gira sobre sí mismo sin desplazarse. Diez arcos pequeños con la misma curvatura dan exactamente la misma pose que un único arco, porque componer arcos de igual radio es equivalente a recorrer un arco más largo.
- **La odometría reconstruye la trayectoria:** primero recta, luego media vuelta de radio $R = v/\omega = 0.6$ m, giro en el sitio y recta final. La pose final queda a 0.2 mm y 0.03° de la ideal: con un encoder de 1024 pulsos, contados sin acumular sesgo, la cuantización es despreciable. En un robot real, los errores importantes de la odometría vienen del deslizamiento de las ruedas, de radios o distancia entre ruedas mal calibrados y de un suelo irregular, no de la resolución del encoder.

Ninguno de esos efectos se simula aquí, y precisamente son los que modelan las covarianzas de las secciones siguientes.

- **Incrementos publicados a 10 Hz.** Cada $\Delta\mathbf{x}_t^{\text{odo}}$ agrupa 10 incrementos del firmware y está expresado en el marco del robot en $t-1$. En los tramos rectos vale prácticamente $[0.04, 0, 0]^T$ (0.4 m/s × 0.1 s), sea cual sea la dirección global del robot. El ± 0.001 rad que aparece en $\Delta\theta$ se debe a que, en un periodo, una rueda puede contar un pulso más que la otra. En el arco aparece una pequeña componente lateral Δy, aunque el robot nunca se desplaza de lado. Esto es lo que recibe como entrada el modelo de movimiento de odometría.

END OF OPTIONAL PART

3.3.1 Analytic form

As we did in Chapter 3.1, the analytic form of the odometry motion model uses the **composition of poses** to model the robot motion. In this formulation, only the pose increment between two odometry readings is considered, without explicitly modeling the trajectory followed between them.

As in the **velocity motion model**, the robot pose is represented by a Gaussian distribution:

$$\mathbf{x}_t \sim \mathcal{N}\left(\bar{\mathbf{x}}_t, \Sigma_{\mathbf{x}_t}\right).$$

The mean and covariance are propagated as follows.

Mean

The mean pose is obtained by composing the previous mean pose with the odometry increment:

$$\bar{\mathbf{x}}_t = g_o\left(\bar{\mathbf{x}}_{t-1}, \Delta\bar{\mathbf{x}}_t^{\text{odo}}\right) = \bar{\mathbf{x}}_{t-1} \oplus \Delta\bar{\mathbf{x}}_t^{\text{odo}},$$

where

$$\Delta\bar{\mathbf{x}}_t^{\text{odo}} = \begin{bmatrix} \Delta x_t \\ \Delta y_t \\ \Delta\theta_t \end{bmatrix}.$$

Therefore,

$$g_o\left(\bar{\mathbf{x}}_{t-1}, \Delta\bar{\mathbf{x}}_t^{\text{odo}}\right) = \begin{bmatrix} \bar{x}_{t-1} + \Delta x_t \cos\bar{\theta}_{t-1} - \Delta y_t \sin\bar{\theta}_{t-1} \\ \bar{y}_{t-1} + \Delta x_t \sin\bar{\theta}_{t-1} + \Delta y_t \cos\bar{\theta}_{t-1} \\ \bar{\theta}_{t-1} + \Delta\theta_t \end{bmatrix}.$$

Covariance

Since the motion model

$$\bar{\mathbf{x}}_t = g_o\left(\bar{\mathbf{x}}_{t-1}, \Delta\bar{\mathbf{x}}_t^{\text{odo}}\right)$$

is nonlinear, the transformed distribution is not exactly Gaussian. We therefore approximate its covariance by linearizing the motion model around the current mean values.

The covariance can be written as

$$\Sigma_{\mathbf{x}_t} \approx \frac{\partial g_o}{\partial \mathbf{x}_{t-1}} \Sigma_{\mathbf{x}_{t-1}} \left(\frac{\partial g_o}{\partial \mathbf{x}_{t-1}}\right)^T + \frac{\partial g_o}{\partial \Delta\mathbf{x}_t^{\text{odo}}} \Sigma_{\Delta\mathbf{x}_t^{\text{odo}}} \left(\frac{\partial g_o}{\partial \Delta\mathbf{x}_t^{\text{odo}}}\right)^T.$$

The Jacobian with respect to the previous robot pose is

$$\frac{\partial g_o}{\partial \mathbf{x}_{t-1}} = \begin{bmatrix} 1 & 0 & -\Delta x_t \sin\theta_{t-1} - \Delta y_t \cos\theta_{t-1} \\ 0 & 1 & \Delta x_t \cos\theta_{t-1} - \Delta y_t \sin\theta_{t-1} \\ 0 & 0 & 1 \end{bmatrix},$$

while the Jacobian with respect to the odometry increment is

$$\frac{\partial g_o}{\partial \Delta\mathbf{x}_t^{\text{odo}}} = \begin{bmatrix} \cos\theta_{t-1} & -\sin\theta_{t-1} & 0 \\ \sin\theta_{t-1} & \cos\theta_{t-1} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Finally, the covariance associated with the odometry increment is modeled as

$$\Sigma_{\Delta\mathbf{x}_t^{\text{odo}}} = \begin{bmatrix} \sigma_{\Delta x}^2 & 0 & 0 \\ 0 & \sigma_{\Delta y}^2 & 0 \\ 0 & 0 & \sigma_{\Delta\theta}^2 \end{bmatrix}.$$

Here, no correlation between the three components of the odometry increment is assumed.

In practice, this covariance may be modeled as constant, although it can also be parameterized as a function of the amount of motion. This is reasonable because larger traveled distances and rotations typically introduce larger odometry uncertainty.

Gentle reminder: Jacobians

Recall that the Jacobian matrix of a vector-valued function

$$\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$$

is defined as

$$\frac{\partial\{f_1,\ldots,f_m\}}{\partial\{x_1,\ldots,x_n\}} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}.$$

Jacobians tell us how small variations or uncertainties in the inputs of the motion model affect the resulting robot pose.

In this case, they serve two main purposes:

- **Linearize the nonlinear motion model:** pose composition involves trigonometric terms, so the exact transformation of a Gaussian uncertainty is not Gaussian. The Jacobians provide a local linear approximation around the current pose and odometry increment.
- **Transform uncertainties into the output pose space:** the previous-pose uncertainty is expressed in the world frame, while the odometry increment uncertainty is expressed in the robot local frame. Their corresponding Jacobians transform both contributions into the same output pose space, where they can be combined.
- Extra: if needed, they adapt covariance matrices so they share the same dimensions and they can be added.

In other words, each Jacobian answers the question:

> *If this input changes slightly, how much will the final pose change?*

ASSIGNMENT 1: The model in action

Similarly to the assignment 3.1, we'll move a robot along an 8-by-8 square (in meters), in increments of 2m. In this case you have to complete:

- The `step()` method to compute:
  - the new expected pose (`self.pose`),
  - the new true pose $\mathbf{x}_t$ (ground-truth `self.true_pose`) after adding some noise using `stats.multivariate_normal.rvs()` to the odometry increment $\mathbf{u}_t = \Delta\mathbf{x}_t^{\text{odo}}$ according to `Q` (which represents $\Sigma_{\Delta\mathbf{x}_t^{\text{odo}}}$),
  - and to update the uncertainty about the robot position in `self.P` (covariance matrix $\Sigma_{\mathbf{x}_t}$). Note that the methods `J1()` and `J2()` already implement $\partial g_o/\partial \mathbf{x}_{t-1}$ and $\partial g_o/\partial \Delta\mathbf{x}_t^{\text{odo}}$ for you, you just have to call them with the right input parameters.
- The `draw()` method to plot:

- the uncertainty of the pose as an ellipse centered at the expected pose, and
- the true position (ground-truth).

We are going to consider the following motion covariance matrix (it is already coded for you):

$$\Sigma_{\Delta \mathbf{x}_t^{\text{odo}}} = \begin{bmatrix} 0.04 & 0 & 0 \\ 0 & 0.04 & 0 \\ 0 & 0 & 0.01 \end{bmatrix}$$

Example

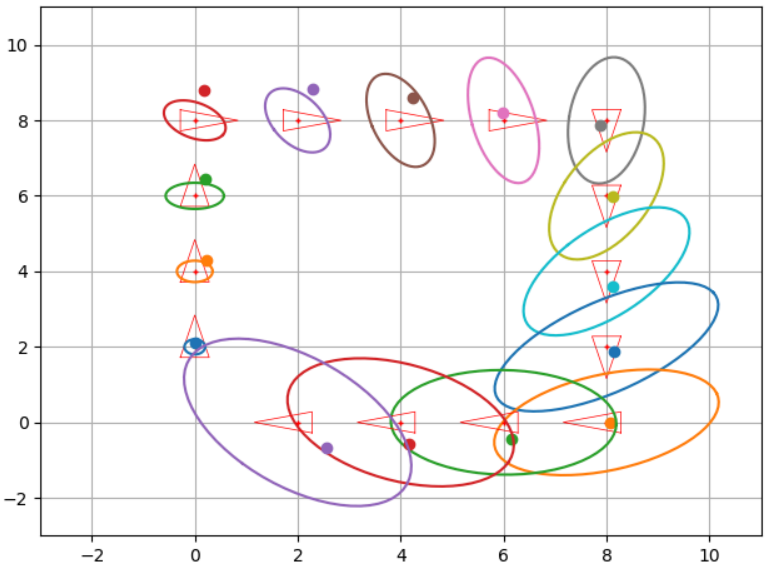


Fig. 3: Movement of a robot using odometry commands. Representing the expected pose (in red), the true pose (as dots) and the confidence ellipse.

In [4]:

```
class Robot():
    """ Simulation of a robot base

        Attrs:
            pose: Expected pose of the robot
            P: Covariance of the current pose
            true_pose: Real pose of the robot(affected by noise)
            Q: Covariance of the movement
    """
    def __init__(self, x, P, Q):
        self.pose = x
        self.P = P
        self.true_pose = self.pose
        self.Q = Q

    def step(self, u):
        # TODO Update expected pose
        prev_pose = self.pose
        self.pose = tcomp(prev_pose, u)
```

```
        # TODO Generate true pose
        noisy_u = np.vstack(stats.multivariate_normal.rvs(mean=u.ravel(),
cov=self.Q))
        self.true_pose = tcomp(self.true_pose, noisy_u)

        # TODO Update covariance
        JacF_x = J1(prev_pose, u)
        JacF_u = J2(prev_pose, u)

        self.P = (
            (JacF_x @ self.P @ JacF_x.T)
            + (JacF_u @ self.Q @ JacF_u.T)
        )

    def draw(self, fig, ax):
        DrawRobot(fig, ax, self.pose)
        el = PlotEllipse(fig, ax, self.pose, self.P)
        ax.plot(self.true_pose[0, 0], self.true_pose[1, 0], 'o',
color=el[0].get_color())
```

You can use the following demo to **try your new `Robot()` class**.

In [5]:
```
def demo_odometry_commands_analytical(robot):
    # MATPLOTLIB
    fig, ax = plt.subplots()
    ax.set_xlim([-3, 11])
    ax.set_ylim([-3, 11])
    plt.ion()
    plt.grid()
    plt.fill([2, 2, 6, 6],[2, 6, 6, 2],facecolor='lightgray',
edgecolor='gray', linewidth=3)
    plt.tight_layout()
    fig.canvas.draw()

    # MOVEMENT PARAMETERS
    nSteps = 15
    ang = -np.pi/2 # angle to turn in corners
    u = np.vstack((2., 0., 0.))

    # MAIN LOOP
    for i in range(nSteps):
        # change angle on corners
        if i % 4 == 3:
            u[2, 0] = ang

        #Update positions
        robot.step(u)

        # Restore angle iff changed
        if i % 4 == 3:
            u[2, 0] = 0

        # Draw every loop
```

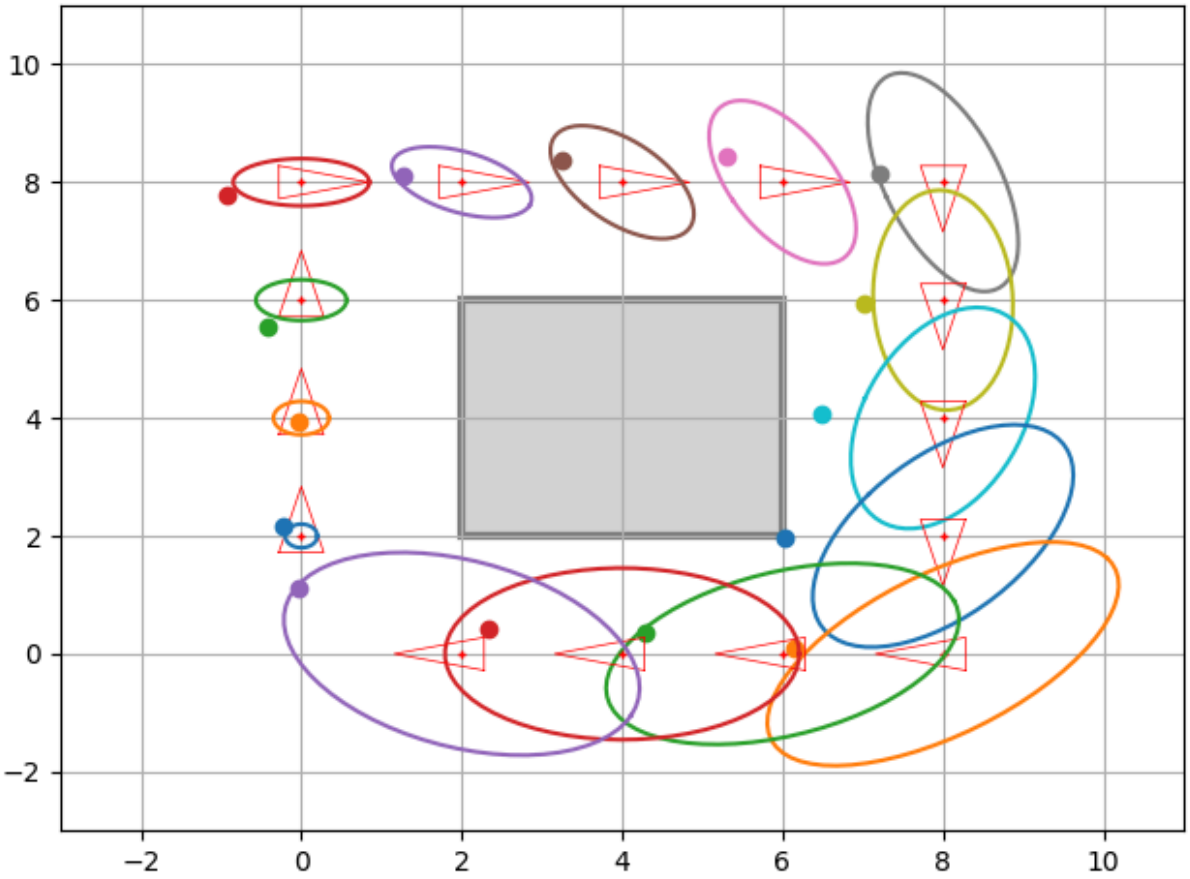
```
        robot.draw(fig, ax)
        clear_output(wait=True)
        display(fig)
        time.sleep(0.3)

    plt.close()
```

In [6]:
```
x = np.vstack([0., 0., np.pi/2]) # initial pose

# Probabilistic parameters
P = np.diag([0., 0., 0.])
Q = np.diag([0.04, 0.04, 0.01])

robot = Robot(x, P, Q)
demo_odometry_commands_analytical(robot)
```



Experimento: distintas matrices $\Sigma_{\Delta \mathbf{x}_t^{\text{odo}}}$ y validación de la covarianza

La última pregunta de *Thinking about it (1)* pide probar distintos valores de $\Sigma_{\Delta \mathbf{x}_t^{\text{odo}}}$ (`Q`). Se repite sin animación el recorrido cuadrado de `demo_odometry_commands_analytical()` (15 incrementos de 2 m, giro a la derecha cada 4), reutilizando la clase `Robot`, con seis matrices:

- **Fila superior:** la del enunciado, $\mathrm{diag}(0.04, 0.04, 0.01)$; diez veces menor, $\mathrm{diag}(0.004, 0.004, 0.001)$; y con mucho ruido angular, $\mathrm{diag}(0.04, 0.04, 0.1)$.
- **Fila inferior:** solo ruido de traslación, $\mathrm{diag}(0.04, 0.04, \approx 0)$; solo ruido de rotación, $\mathrm{diag}(\approx 0, \approx 0, 0.01)$; y solo ruido longitudinal (en la dirección de avance), $\mathrm{diag}(0.04, \approx 0, \approx 0)$. Las varianzas «nulas» valen 10^{-12} porque `multivariate_normal` necesita una matriz no singular.

En cada figura se dibujan la pose esperada (rojo), la elipse de 1σ de $\Sigma_{\mathbf{x}_t}$ y la pose real (punto) tras cada paso. Se fija la semilla para que los resultados comentados sean reproducibles.

Después, una simulación de Monte Carlo comprueba si la covarianza propagada con los jacobianos es correcta: se repite el recorrido 2000 veces con la `Q` del enunciado y se compara la nube de poses reales finales con la elipse de $\Sigma_{\mathbf{x}_t}$. Si la aproximación lineal fuese exacta, el 39.3 %, 86.5 % y 98.9 % de los puntos caerían dentro de las elipses de 1, 2 y 3 σ, respectivamente (distribución χ^2 con 2 grados de libertad).

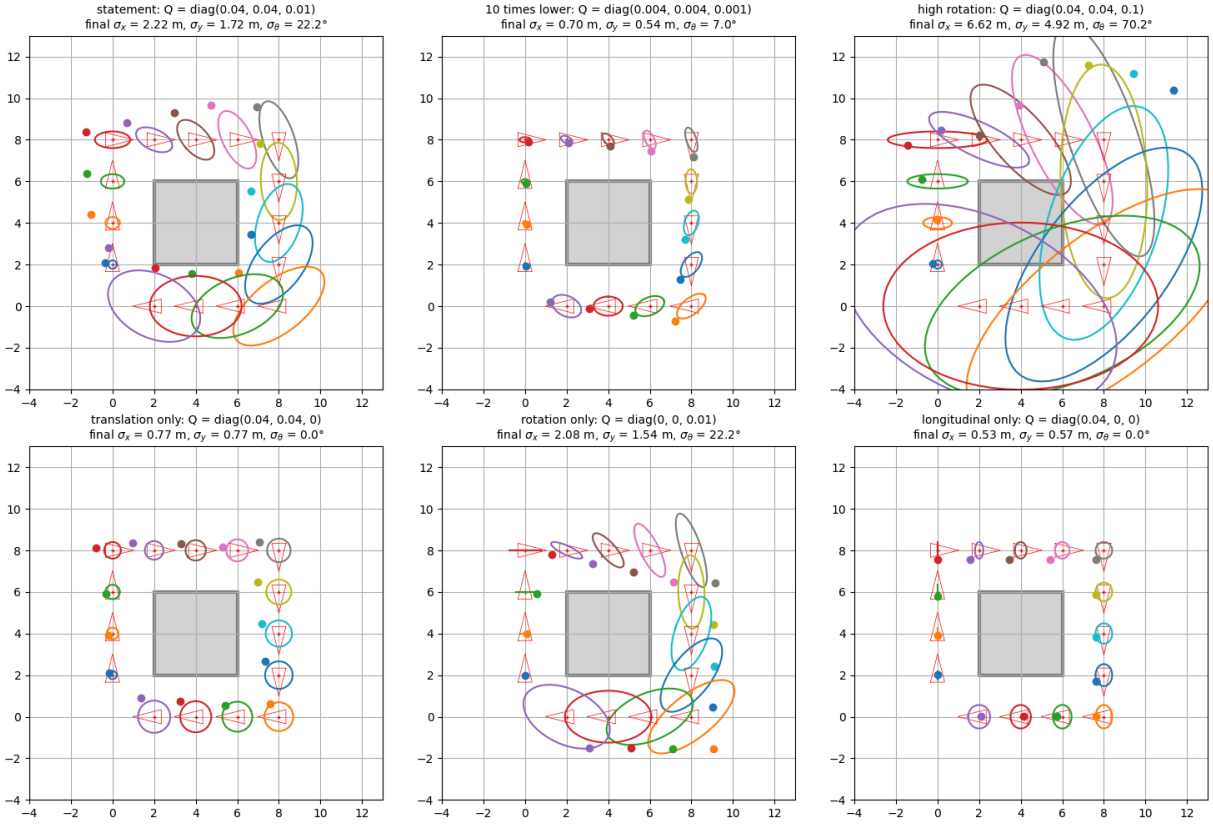
In [7]:
```
def simulate_square(robot, fig=None, ax=None, n_steps=15, ang=-np.pi/2):
    '''Runs the square path of demo_odometry_commands_analytical() without
animation (draws if ax is given).'''
    u = np.vstack((2., 0., 0.))
    for i in range(n_steps):
        u[2, 0] = ang if i % 4 == 3 else 0
        robot.step(u)
        if ax is not None:
            robot.draw(fig, ax)
    return robot

eps = 1e-12
Qs = [
    ('statement', np.diag([0.04, 0.04, 0.01])),
    ('10 times lower', np.diag([0.004, 0.004, 0.001])),
    ('high rotation', np.diag([0.04, 0.04, 0.1])),
    ('translation only', np.diag([0.04, 0.04, eps])),
    ('rotation only', np.diag([eps, eps, 0.01])),
    ('longitudinal only', np.diag([0.04, eps, eps])),
]
x0, P0 = np.vstack([0., 0., np.pi/2]), np.zeros((3, 3))
np.random.seed(0)

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
print(f"{'Q':>18} | final sigma_x, sigma_y (m), sigma_theta (deg) | true
position error (m)")
for ax, (name, Q) in zip(axes.ravel(), Qs):
    plt.sca(ax)  # DrawRobot sizes the robot using the current axes
    ax.set(xlim=(-4, 13), ylim=(-4, 13), aspect='equal')
    ax.grid(True)
    ax.fill([2, 2, 6, 6], [2, 6, 6, 2], facecolor='lightgray',
edgecolor='gray', linewidth=3)
    robot = simulate_square(Robot(x0, P0, Q), fig, ax)
    sx, sy, st = np.sqrt(np.diag(robot.P))
    error = np.linalg.norm(robot.pose[:2] - robot.true_pose[:2])
```

```
    variances = ', '.join(f'{v:g}' if v > eps else '0' for v in np.diag(Q))
    ax.set_title(f"{name}: Q = diag({variances})\nfinal $\\sigma_x$ =
{sx:.2f} m, $\\sigma_y$ = {sy:.2f} m, "
                 f"$\\sigma_\\theta$ = {np.degrees(st):.1f}°", fontsize=10)
    print(f"{name:>18} | {sx:6.2f} {sy:6.2f} {np.degrees(st):8.1f}
| {error:6.2f}")
plt.tight_layout()
plt.show()
```

```
                 Q | final sigma_x, sigma_y (m), sigma_theta (deg) | true
position error (m)
         statement |   2.22   1.72     22.2                        |   3.56
    10 times lower |   0.70   0.54      7.0                        |   0.83
      high rotation |   6.62   4.92     70.2                        |  13.32
  translation only |   0.77   0.77      0.0                        |   1.10
     rotation only |   2.08   1.54     22.2                        |   1.84
 longitudinal only |   0.53   0.57      0.0                        |   0.10
```



In [8]:
```
# Monte Carlo validation of the covariance propagated with the Jacobians
(2000 runs, statement Q)
n_runs = 2000
Q = np.diag([0.04, 0.04, 0.01])
np.random.seed(0)
finals = np.hstack([simulate_square(Robot(x0, P0, Q)).true_pose for _ in
range(n_runs)])
robot = simulate_square(Robot(x0, P0, Q))  # its mean and P are
deterministic: they do not depend on the noise

err = finals - robot.pose
err[2] = np.arctan2(np.sin(err[2]), np.cos(err[2]))
print("Predicted covariance P (Jacobians):\n", robot.P.round(3))
print(f"\nSample covariance of the {n_runs} true final poses:\n",
```

```
np.cov(err).round(3))
print(f"\nMean of the true final poses: {finals[:2].mean(axis=1).round(2)}
vs. expected pose {robot.pose[:2, 0].round(2)}")

mahalanobis2 = np.sum(err[:2] * (np.linalg.inv(robot.P[:2, :2]) @ err[:2]),
axis=0)
print("\nFraction of true final positions inside the ellipse (x-y):")
for n_sigma, gaussian in zip((1, 2, 3), (39.3, 86.5, 98.9)):
    print(f"  {n_sigma} sigma: {100 * np.mean(mahalanobis2 <=
n_sigma**2):5.1f} %   (Gaussian: {gaussian} %)")

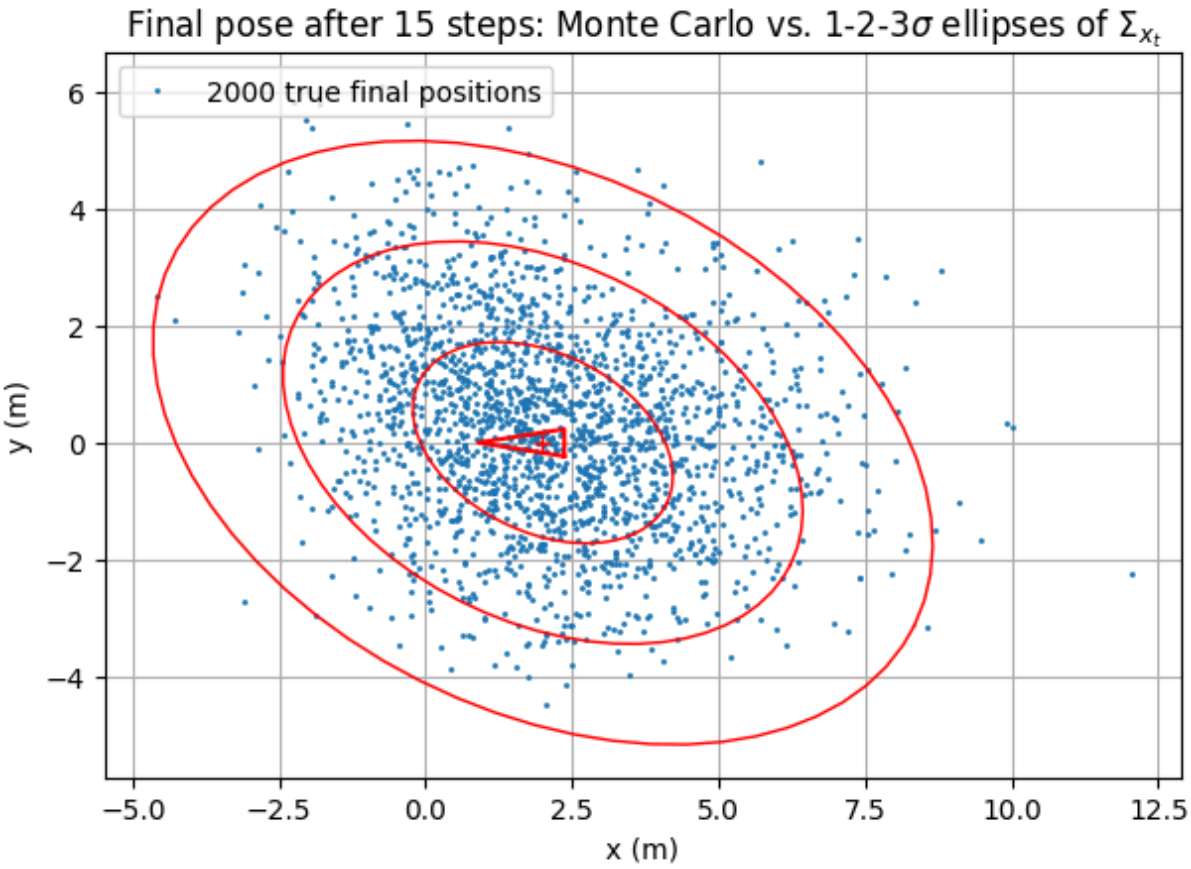
fig, ax = plt.subplots(figsize=(7, 7))
plt.sca(ax)
ax.plot(finals[0], finals[1], '.', markersize=2, color='tab:blue',
label=f'{n_runs} true final positions')
DrawRobot(fig, ax, robot.pose, linewidth=1.5)
for n_sigma in (1, 2, 3):
    PlotEllipse(fig, ax, robot.pose, robot.P, n_sigma, color='red',
linewidth=1)
ax.set(aspect='equal', xlabel='x (m)', ylabel='y (m)',
       title='Final pose after 15 steps: Monte Carlo vs. 1-2-3$\\sigma$
ellipses of $\\Sigma_{x_t}$')
ax.grid(True)
ax.legend(loc='upper left')
plt.show()
```

```
Predicted covariance P (Jacobians):
 [[ 4.92 -1.28  0.64]
 [-1.28  2.96 -0.34]
 [ 0.64 -0.34  0.15]]

Sample covariance of the 2000 true final poses:
 [[ 4.733 -0.9    0.6  ]
 [-0.9    2.952 -0.277]
 [ 0.6   -0.277  0.145]]

Mean of the true final poses: [2.12 0.31]  vs. expected pose [ 2. -0.]

Fraction of true final positions inside the ellipse (x-y):
  1 sigma:  39.2 %   (Gaussian: 39.3 %)
  2 sigma:  85.9 %   (Gaussian: 86.5 %)
  3 sigma:  98.2 %   (Gaussian: 98.9 %)
```



Observaciones:

- **La incertidumbre crece en cada paso.** $\Sigma_{\mathbf{x}_t}$ parte de cero, porque la pose inicial se conoce, y en cada paso se le suma la contribución de `Q`. No hay ninguna medida externa que la reduzca. Con la `Q` del enunciado, tras 15 pasos $\sigma_x \approx 2.2$ m, $\sigma_y \approx 1.7$ m y $\sigma_\theta \approx 22^\circ$.
- $\Sigma_{\mathbf{x}_t}$ **es proporcional a** `Q`**.** Como $\Sigma_{\mathbf{x}_0} = \mathbf{0}$ y los jacobianos no dependen del ruido, dividir `Q` entre 10 divide las desviaciones típicas entre $\sqrt{10} \approx 3.2$ (de 2.22 a 0.70 m y de 22.2° a 7.0°).
- **El ruido angular domina y orienta las elipses.** Solo con ruido de rotación, el σ final (2.1 / 1.5 m) es casi igual que con la `Q` completa. Además, las elipses se alargan **perpendicularmente** a la dirección de avance: es la tercera columna de $\partial g_o / \partial \mathbf{x}_{t-1}$, que convierte un error de orientación en un error lateral proporcional a la distancia recorrida. Tras el primer paso no hay elipse visible, porque el error de giro todavía no ha afectado a ningún avance. En el resto del primer tramo la elipse es casi una recta perpendicular a la marcha.
- **Solo traslación:** con $\sigma_{\Delta x} = \sigma_{\Delta y}$ las elipses son círculos, porque la rotación de $\partial g_o / \partial \Delta\mathbf{x}$ no les cambia la forma. El radio crece como $\sqrt{n}$: $0.2\sqrt{15} \approx 0.77$ m. **Solo longitudinal:** en el primer tramo las elipses se alargan en la dirección de avance. Tras cada esquina conservan el alargamiento heredado de los tramos anteriores, y a él se suma el del nuevo tramo.

- **Con mucho ruido angular** ($\sigma_\theta^2 = 0.1$, unos 18° por paso), σ_θ final llega a 70°. Las elipses son enormes y dejan de tener sentido: con errores de orientación tan grandes, la linealización ya no es válida.
- **Validación de Monte Carlo.** Con la `Q` del enunciado, la covarianza muestral de 2000 trayectorias reales es muy parecida a la $\Sigma_{\mathbf{x}_t}$ propagada con los jacobianos (por ejemplo, 4.73 frente a 4.92 m² en x y 2.95 frente a 2.96 m² en y). Además, el 39.2 %, 85.9 % y 98.2 % de las poses caen dentro de las elipses de 1, 2 y 3 σ, frente al 39.3 %, 86.5 % y 98.9 % teóricos. La aproximación lineal funciona bien para este nivel de ruido. Solo se aprecian pequeñas desviaciones, como colas algo más pesadas y una media real desplazada unos 0.3 m respecto a la esperada, debidas a la no linealidad de la composición.

Thinking about it (1)

Once you have completed this assignment regarding the analytical form of the odometry model, **answer the following questions**:

- Which is the difference between the $g_o(\cdot)$ function used here, and the one in the velocity model, $g_v(\cdot)$?

  *Las dos terminan componiendo la pose anterior con un incremento local, $\boldsymbol{x}_t = \boldsymbol{x}_{t-1} \oplus \Delta\boldsymbol{x}_t$. La diferencia está en de dónde sale ese incremento. En $g_v(\boldsymbol{x}_{t-1}, \boldsymbol{u}_t, \Delta t)$, la entrada es una **orden** de velocidades (v, ω) previa al movimiento. El incremento se **calcula** suponiendo que v y ω se mantienen constantes durante Δt, es decir, que el robot recorre un arco de radio R = v/ω, y hace falta distinguir el caso ω = 0. En $g_o(\boldsymbol{x}_{t-1}, \Delta\boldsymbol{x}_t^{odo})$, en cambio, el incremento ya viene dado: es una **medida** de los encoders, obtenida después del movimiento como $(\ominus\boldsymbol{x}_{t-1}^{odo}) \oplus \boldsymbol{x}_t^{odo}$, y se compone directamente. Por eso g_o no necesita Δt ni casos especiales, ni supone ninguna trayectoria entre las dos lecturas (solo importan la pose inicial y la final). Además, al medir lo que el robot ha hecho de verdad, es más precisa para seguir su pose, aunque solo está disponible a posteriori. g_v sirve para planificar, antes de moverse.*

- How many parameters compound the motion command $\mathbf{u}_t = \Delta\mathbf{x}_t^{\text{odo}}$ in this model?

  Tres: Δx_t, Δy_t y $\Delta\theta_t$, expresados en el marco local del robot en t-1. Es uno más que en el modelo de velocidad (v, ω). La componente lateral Δy no significa que el robot se mueva de lado, sino que, tras un arco, el punto final queda desplazado lateralmente respecto a la orientación inicial.

- Which is the role of the Jacobians $\partial g_o/\partial\mathbf{x}_{t-1}$ and $\partial g_o/\partial\Delta\mathbf{x}_t^{\text{odo}}$?

  Sirven para propagar la incertidumbre. La composición tiene senos y cosenos de θ, así que no es lineal y una gaussiana no se transforma en otra gaussiana. Los

jacobianos son la mejor aproximación lineal de g_o alrededor de la media y permiten calcular $\Sigma_{\boldsymbol{x}t} \approx J_x \Sigma_{\boldsymbol{x}t-1} J_x^T + J_u \Sigma_{\Delta\boldsymbol{x}} J_u^T$, como en el filtro de Kalman extendido. Cada uno responde a «si esta entrada cambia un poco, ¿cuánto cambia la pose final?». $\partial g_o / \partial \boldsymbol{x}_{t-1}$ lleva la incertidumbre de la pose anterior a la nueva pose. Su tercera columna convierte un error de orientación en un error de posición proporcional al avance (efecto palanca), y por eso las elipses se alargan perpendicularmente a la marcha. $\partial g_o / \partial \Delta\boldsymbol{x}^{odo}$ es la matriz de rotación $R(\theta_{t-1})$: pasa el ruido de la odometría, expresado en el marco local, al marco global. Así las dos contribuciones quedan en el mismo espacio y con la misma dimensión, y se pueden sumar. Monte Carlo confirma que la aproximación es buena: la covarianza muestral de 2000 trayectorias coincide con la propagada.

- What happens if you modify the covariance matrix $\Sigma_{\Delta \mathbf{x}_t^{\text{odo}}}$ modeling the uncertainty in the motion command $\mathbf{u}_t = \Delta \mathbf{x}_t^{\text{odo}}$? Try different values and discuss the results.

  $\Sigma_{\Delta\boldsymbol{x}}$ es lo que se suma a la incertidumbre en cada paso, así que controla lo rápido que crecen las elipses y lo lejos que acaba la pose real de la esperada. En el experimento, $\Sigma_{\boldsymbol{x}t}$ resulta proporcional a $\Sigma_{\Delta\boldsymbol{x}}$ (porque $\Sigma_{\boldsymbol{x}0} = 0$): dividirla entre 10 reduce los σ finales en un factor $\sqrt{10}$, de 2.2 a 0.7 m y de 22° a 7°. Cada término influye de forma distinta. Con solo ruido de traslación isótropo, las elipses son círculos que crecen como $\sqrt{n}$ (0.77 m tras 15 pasos). Con solo ruido longitudinal, se alargan en la dirección de avance. Con solo ruido angular, el error final es casi tan grande como con la matriz completa (2.1 frente a 2.2 m), y las elipses se alargan perpendicularmente a la trayectoria, porque cada error de giro desvía todos los avances posteriores. Con mucho ruido angular ($\sigma_\theta^2 = 0.1$), σ_θ llega a 70° y las elipses pierden sentido, porque la linealización deja de ser válida. Si la matriz es demasiado pequeña, el robot es «optimista»: la pose real queda fuera de la elipse. Si es demasiado grande, la estimación es inútil. Debe calibrarse con datos reales.

3.3.2 Sample form

The analytical form used above, although useful for the probabilistic algorithms we will cover in this course, does not work well for sampling algorithms such as particle filters for localization.

The reason being, if we generate random samples from the Gaussian distributions as in the previous exercise, we will find some poses that are not feasible to the non-holonomic movement of a robot, i.e. they do not correspond to a velocity command (v, ω) with noise.

The following *sample form* is a more realistic way to generate samples of the robot pose. In this case, the movement of the robot is modeled as a sequence of actions (see Fig. 4):

1. **Turn** (θ_1): to face the destination point.
2. **Advance** (d): to arrive at the destination.
3. **Turn** (θ_2): to get to the desired angle.

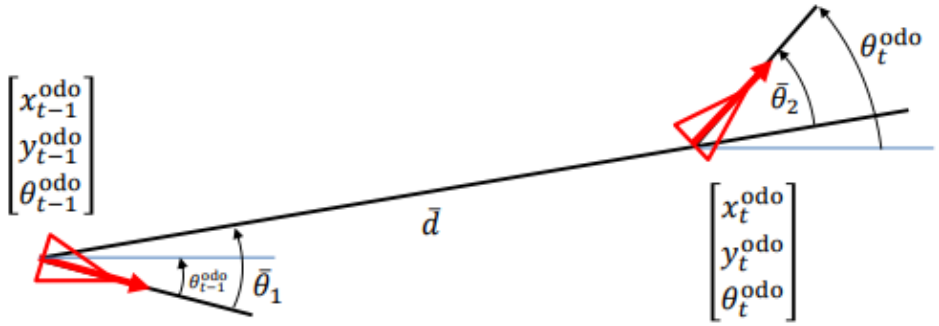


Fig. 4: Movement of a robot using odometry commands in sampling form.

The odometry increment can also be represented as an equivalent sequence of an initial rotation, a straight-line translation, and a final rotation:

$$\mathbf{u}_t^{\text{odo}} = \begin{bmatrix} \bar{\theta}_1 \\ \bar{d} \\ \bar{\theta}_2 \end{bmatrix}$$

These components can be computed from two consecutive odometry poses $\mathbf{x}_{t-1}^{\text{odo}}$ and $\mathbf{x}_t^{\text{odo}}$ as follows:

$$\bar{\theta}_1 = \text{atan2}(y_t^{\text{odo}} - y_{t-1}^{\text{odo}}, x_t^{\text{odo}} - x_{t-1}^{\text{odo}}) - \theta_{t-1}^{\text{odo}}$$

$$\bar{d} = \sqrt{(x_t^{\text{odo}} - x_{t-1}^{\text{odo}})^2 + (y_t^{\text{odo}} - y_{t-1}^{\text{odo}})^2}$$

$$\bar{\theta}_2 = \theta_t^{\text{odo}} - \theta_{t-1}^{\text{odo}} - \bar{\theta}_1$$

This model works by considering a set of possible robot poses $\{\mathbf{x}_{t-1}^{[i]}\}_{i=1}^N$. When a new odometry reading becomes available, the corresponding motion parameters $\mathbf{u}_t^{\text{odo}}$ are computed from two consecutive odometry poses.

Then, the motion model $\mathbf{x}_t^{[i]} = g_o(\mathbf{x}_{t-1}^{[i]}, \mathbf{u}_t^{\text{odo}})$ is used to obtain the new set of poses $\{\mathbf{x}_t^{[i]}\}_{i=1}^N$, by applying a perturbed version of the odometry motion parameters to each particle, as we will see next.

ASSIGNMENT 2: Implementing the sampling form

Complete the following cells to experience the motion of a robot using the sampling form of the odometry model. For that:

1. Implement a function that, given the previously mentioned $\mathbf{x}_t^{\text{odo}}$ and $\mathbf{x}_{t-1}^{\text{odo}}$ generates an order $\mathbf{u}_t^{\text{odo}} = [\bar{\theta}_1, \bar{d}, \bar{\theta}_2]^T$.

In [9]:
```python
def generate_move(pose_now, pose_old):
    diff = pose_now - pose_old
    theta1 = np.arctan2(diff[1], diff[0]) - pose_old[2]
    d = np.sqrt(diff[0]**2 + diff[1]**2)
    theta2 = diff[2] - theta1
    return np.vstack((theta1, d, theta2))
```

Try such function with the code cell below:

In [10]:
```python
generate_move(np.vstack([0., 0., 0.]), np.vstack([1., 1., np.pi/2]))
```

Out[10]:
```
array([[-3.92699082],
       [ 1.41421356],
       [ 2.35619449]])
```

Expected output for the commented example:

```
array([[-3.92699082],
       [ 1.41421356],
       [ 2.35619449]])
```

2. Using the resulting odometry motion parameters $\mathbf{u}_t^{\text{odo}} = [\bar{\theta}_1, \bar{d}, \bar{\theta}_2]^T$, we model their uncertainty by assuming that each component follows a **Gaussian distribution**:

$$\theta_1 \sim \mathcal{N}(\bar{\theta}_1, \sigma_{\theta_1}^2)$$

$$d \sim \mathcal{N}(\bar{d}, \sigma_d^2)$$

$$\theta_2 \sim \mathcal{N}(\bar{\theta}_2, \sigma_{\theta_2}^2)$$

Rather than using constant variances, we compute them as a function of the **amount of motion** and a set of parameters that characterize the robot's intrinsic noise:

$$\sigma_{\theta_1}^2 = \alpha_0 \bar{\theta}_1^2 + \alpha_1 \bar{d}^2$$

$$\sigma_d^2 = \alpha_2 \bar{d}^2 + \alpha_3 (\bar{\theta}_1^2 + \bar{\theta}_2^2)$$

$$\sigma_{\theta_2}^2 = \alpha_0 \bar{\theta}_2^2 + \alpha_1 \bar{d}^2$$

where $\boldsymbol{\alpha} = [\alpha_0, \ldots, \alpha_3]^T$ (`a` in the code) contains the robot's intrinsic noise parameters. Thus, larger translations or rotations generally introduce greater uncertainty.

The noisy motion parameters can then be generated **independently for each sample** $i = 1, \ldots, N$ as:

$$\theta_1^{[i]} = \bar{\theta}_1 + \text{sample}(\sigma_{\theta_1}^2)$$

$$d^{[i]} = \bar{d} + \text{sample}(\sigma_d^2)$$

$$\theta_2^{[i]} = \bar{\theta}_2 + \text{sample}(\sigma_{\theta_2}^2)$$

where $\text{sample}(b^2)$ generates an independent random value from $\mathcal{N}(0, b^2)$ for each particle, with b^2 representing the variance. These are the components of the noisy motion $\tilde{\mathbf{u}}_t^{\text{odo},[i]} = [\theta_1^{[i]}, d^{[i]}, \theta_2^{[i]}]^T$

Thus, each particle receives a different realization of the same odometry motion, according to the uncertainty of its three components.

The motion model $\mathbf{x}_t^{[i]} = g_o(\mathbf{x}_{t-1}^{[i]}, \tilde{\mathbf{u}}_t^{\text{odo},[i]})$ is then applied to each particle using its corresponding sampled motion parameters:

$$x_t^{[i]} = x_{t-1}^{[i]} + d^{[i]} \cos(\theta_{t-1}^{[i]} + \theta_1^{[i]})$$

$$y_t^{[i]} = y_{t-1}^{[i]} + d^{[i]} \sin(\theta_{t-1}^{[i]} + \theta_1^{[i]})$$

$$\theta_t^{[i]} = \theta_{t-1}^{[i]} + \theta_1^{[i]} + \theta_2^{[i]}$$

Complete the `step()` and `draw()` methods to:

- Update the expected robot pose (`self.pose`) and generate new samples. The number of samples is set by `n_samples` , and `self.samples` is in charge of storing such samples. Each sample can be interpreted as one possible pose reached by the robot.
- Draw the true pose of the robot (without angle) as a cloud of particles (samples of possible points which the robot can be at).

Play a bit with different values of `a` . To improve this visualization the robot will move in increments of 0.5 and we are going to plot the particles every 4 increments.

Example

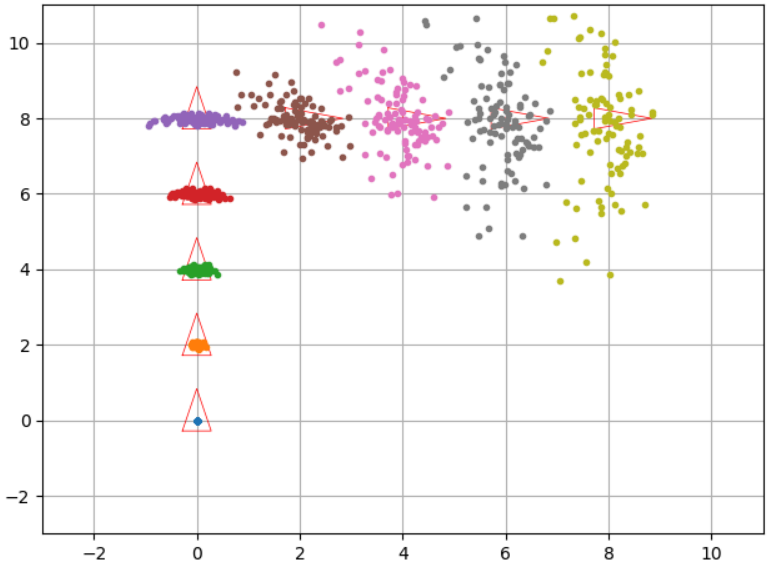


Fig. 5: Movement of a robot using odometry commands in sampling form. Representing the expected pose (in red) and the samples (as clouds of dots)

In [11]:

```
class SampledRobot(object):
    def __init__(self, mean, a, n_samples):
        self.pose = mean
        self.a = a
        self.samples = np.tile(mean, n_samples)

    def step(self, u):
        # TODO Update pose
        ang = self.pose[2, 0] + u[0, 0]
        self.pose[0, 0] += u[1, 0]*np.cos(ang)
        self.pose[1, 0] += u[1, 0]*np.sin(ang)
        self.pose[2, 0] = ang + u[2, 0]

        # TODO Generate new samples
        # b2 is the variance; scipy's scale expects the standard deviation
        sample = lambda b2: stats.norm(loc=0,
scale=np.sqrt(b2)).rvs(size=self.samples.shape[1])

        u2 = u**2

        noisy_u = u + np.vstack((
            sample(self.a[0]*u2[0] + self.a[1]*u2[1]),
            sample(self.a[2]*u2[1] + self.a[3]*(u2[0] + u2[2])),
            sample(self.a[0]*u2[2] + self.a[1]*u2[1])
        ))

        # TODO Update particles (robots) poses
        ang = self.samples[2, :] + noisy_u[0, :]

        self.samples[0, :] += noisy_u[1, :]*np.cos(ang)
        self.samples[1, :] += noisy_u[1, :]*np.sin(ang)
        self.samples[2, :] = ang + noisy_u[2, :]
```

```
    def draw(self, fig, ax):
        DrawRobot(fig, ax, self.pose)
        ax.plot(self.samples[0, :], self.samples[1, :], '.')
```

Run the following demo to **test your code**:

In [12]:
```
def demo_odometry_commands_sample(robot):
    # PARAMETERS
    inc = .5
    show_each = 4
    limit_iterations = 32

    # MATPLOTLIB
    fig, ax = plt.subplots()
    ax.set_xlim([-3, 11])
    ax.set_ylim([-3, 11])
    plt.ion()
    plt.grid()
    plt.tight_layout()

    # MAIN LOOP
    robot.draw(fig, ax)
    inc_pose = np.vstack((0., inc, 0.))

    for i in range(limit_iterations):
        if i == 16:
            inc_pose[0, 0] = inc
            inc_pose[1, 0] = 0
            inc_pose[2, 0] = -np.pi/2

        u = generate_move(robot.pose+inc_pose, robot.pose)

        robot.step(u)

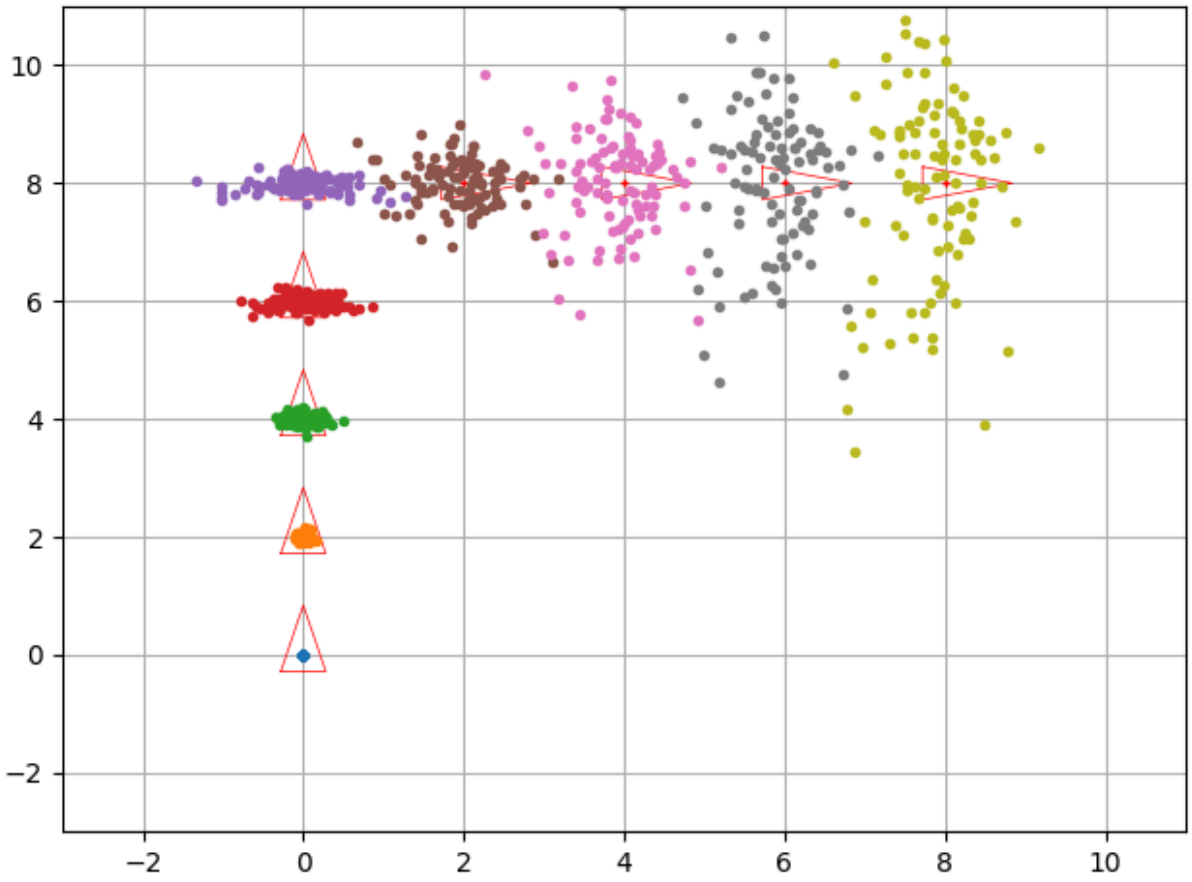
        if i == 16:
            inc_pose[2, 0] = 0

        if i % show_each == show_each-1:
            robot.draw(fig, ax)
            clear_output(wait=True)
            display(fig)
            time.sleep(0.1)

    plt.close()
```

In [13]:
```
# RUN
n_particles = 100
a = np.array([.015, .0012, .003, .007])
x = np.vstack((0., 0., np.pi/2))

robot = SampledRobot(x, a, n_particles)
demo_odometry_commands_sample(robot)
```



Experimento: efecto de los parámetros de ruido α

El enunciado pide jugar con distintos valores de `a`. Se repite sin animación el mismo recorrido de `demo_odometry_commands_sample()` (32 incrementos de 0.5 m, con giro a la derecha en el incremento 16; se dibujan las muestras cada 4 incrementos), reutilizando `generate_move()` y `SampledRobot`, con 200 partículas:

- **Fila superior:** los α del enunciado, $[0.015, 0.0012, 0.003, 0.007]$, multiplicados por 0.1, 1, 4 y 10.
- **Fila inferior:** cada vez solo uno de los cuatro parámetros con su valor del enunciado (los demás «desactivados»), para aislar su papel: α_0 (giro → error de giro), α_1 (traslación → error de giro), α_2 (traslación → error de traslación) y α_3 (giro → error de traslación). Un parámetro «desactivado» vale 10^{-10} porque `stats.norm` necesita una desviación típica estrictamente positiva.

En el título de cada figura se indica la dispersión final de la nube (desviación típica en x e y). Después, una segunda celda mueve 1000 partículas 10 m en línea recta para ver con claridad **qué forma** toma la distribución. Se fija la semilla para que los resultados comentados sean reproducibles.

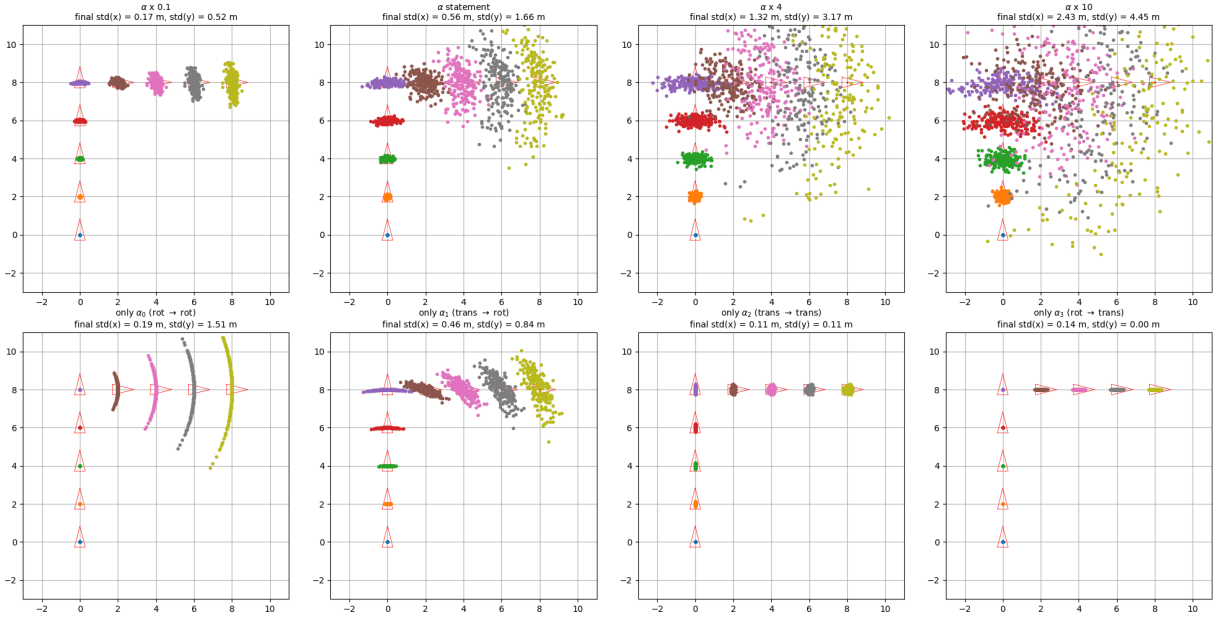
In [14]:
```
def simulate_sampled(robot, fig, ax, inc=.5, show_each=4,
limit_iterations=32):
    '''Runs the path of demo_odometry_commands_sample() without animation,
```

```
drawing the samples every show_each steps.'''
    plt.sca(ax)  # DrawRobot sizes the robot using the current axes
    robot.draw(fig, ax)
    inc_pose = np.vstack((0., inc, 0.))
    for i in range(limit_iterations):
        if i == 16:
            inc_pose = np.vstack((inc, 0., -np.pi/2))
        robot.step(generate_move(robot.pose + inc_pose, robot.pose))
        if i == 16:
            inc_pose[2, 0] = 0
        if i % show_each == show_each - 1:
            robot.draw(fig, ax)
    return robot

eps = 1e-10
a_ref = np.array([.015, .0012, .003, .007])
only = lambda k: np.where(np.arange(4) == k, a_ref, eps)
plain = lambda s: s.replace('$', '').replace('\\to', '->').replace('\\',
'')
alphas = [
    ('$\\alpha$ x 0.1', a_ref * 0.1), ('$\\alpha$ statement', a_ref),
    ('$\\alpha$ x 4', a_ref * 4), ('$\\alpha$ x 10', a_ref * 10),
    ('only $\\alpha_0$ (rot $\\to$ rot)', only(0)), ('only $\\alpha_1$
(trans $\\to$ rot)', only(1)),
    ('only $\\alpha_2$ (trans $\\to$ trans)', only(2)), ('only $\\alpha_3$
(rot $\\to$ trans)', only(3)),
]
np.random.seed(0)

fig, axes = plt.subplots(2, 4, figsize=(20, 10))
print(f"{'alpha':>28} | final spread std(x), std(y) (m)")
for ax, (name, a) in zip(axes.ravel(), alphas):
    ax.set(xlim=(-3, 11), ylim=(-3, 11), aspect='equal')
    ax.grid(True)
    robot = simulate_sampled(SampledRobot(np.vstack((0., 0., np.pi/2)), a,
200), fig, ax)
    std_x, std_y = robot.samples[:2].std(axis=1)
    ax.set_title(f"{name}\nfinal std(x) = {std_x:.2f} m, std(y) =
{std_y:.2f} m", fontsize=10)
    print(f"{plain(name):>28} | {std_x:6.2f} {std_y:6.2f}")
plt.tight_layout()
plt.show()
```

```
                      alpha | final spread std(x), std(y) (m)
                alpha x 0.1 |   0.17   0.52
            alpha statement |   0.56   1.66
                  alpha x 4 |   1.32   3.17
                 alpha x 10 |   2.43   4.45
  only alpha_0 (rot -> rot) |   0.19   1.51
only alpha_1 (trans -> rot) |   0.46   0.84
only alpha_2 (trans -> trans) |   0.11   0.11
only alpha_3 (rot -> trans) |   0.14   0.00
```

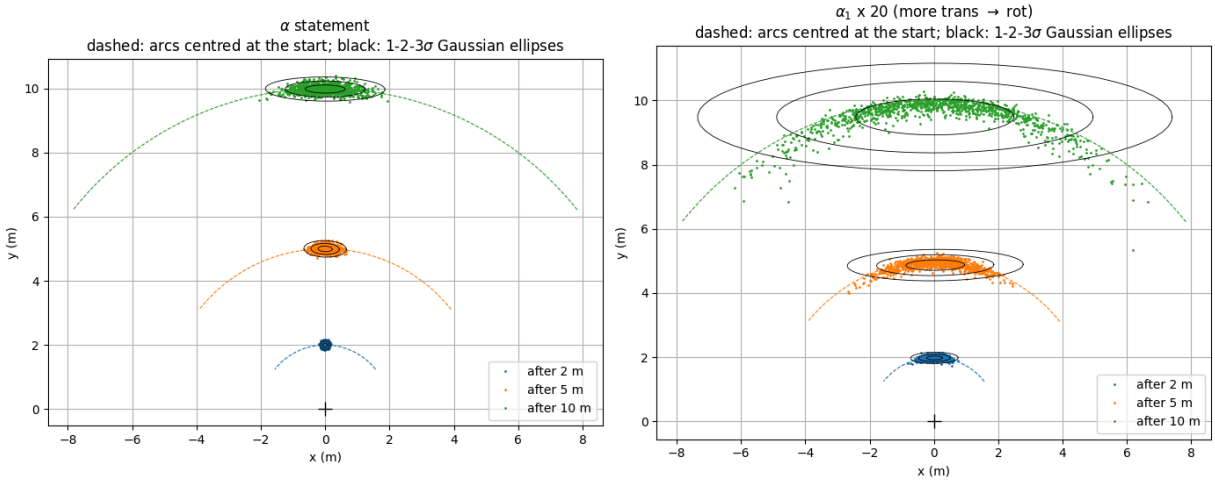


In [15]:
```
# Shape of the distribution: 1000 particles moving 10 m straight ahead (20
increments of 0.5 m)
def straight_run(a, n_particles=1000, inc=.5, n_steps=20, keep=(4, 10,
20)):
    robot = SampledRobot(np.vstack((0., 0., np.pi/2)), a, n_particles)
    clouds = {}
    for k in range(1, n_steps + 1):
        robot.step(generate_move(robot.pose + np.vstack((0., inc, 0.)),
robot.pose))
        if k in keep:
            clouds[k * inc] = robot.samples.copy()
    return clouds

np.random.seed(1)
cases = [('$\\alpha$ statement', a_ref), ('$\\alpha_1$ x 20 (more trans
$\\to$ rot)', a_ref * np.array([1, 20, 1, 1]))]
fig, axes = plt.subplots(1, 2, figsize=(14, 7))
for ax, (name, a) in zip(axes, cases):
    clouds = straight_run(a)
    for dist, cloud in clouds.items():
        pts = ax.plot(cloud[0], cloud[1], '.', markersize=2, label=f'after
{dist:g} m')
        phi = np.linspace(np.pi/2 - 0.9, np.pi/2 + 0.9, 100)
        ax.plot(dist * np.cos(phi), dist * np.sin(phi), '--',
color=pts[0].get_color(), linewidth=0.8)
        mean, cov = cloud[:2].mean(axis=1, keepdims=True),
np.cov(cloud[:2])
        for n_sigma in (1, 2, 3):
            PlotEllipse(fig, ax, mean, cov, n_sigma, color='black',
linewidth=0.6)
        r = np.linalg.norm(cloud[:2], axis=0)
        print(f"{plain(name):>32} | after {dist:4g} m: std(x) =
{cloud[0].std():.2f} m (lateral), "
              f"std(distance to start) = {r.std():.2f} m, std(theta) =
{np.degrees(cloud[2].std()):.1f} deg")
    ax.plot(0, 0, 'k+', markersize=12)
    ax.set(aspect='equal', title=f"{name}\ndashed: arcs centred at the
```

```
start; black: 1-2-3$\\sigma$ Gaussian ellipses",
           xlabel='x (m)', ylabel='y (m)')
    ax.grid(True)
    ax.legend(loc='lower right')
plt.tight_layout()
plt.show()
```

```
                alpha statement | after    2 m: std(x) = 0.06 m (lateral),
std(distance to start) = 0.05 m, std(theta) = 2.8 deg
                alpha statement | after    5 m: std(x) = 0.22 m (lateral),
std(distance to start) = 0.09 m, std(theta) = 4.4 deg
                alpha statement | after   10 m: std(x) = 0.62 m (lateral),
std(distance to start) = 0.12 m, std(theta) = 6.3 deg
alpha_1 x 20 (more trans -> rot) | after    2 m: std(x) = 0.25 m (lateral),
std(distance to start) = 0.05 m, std(theta) = 12.1 deg
alpha_1 x 20 (more trans -> rot) | after    5 m: std(x) = 0.91 m (lateral),
std(distance to start) = 0.10 m, std(theta) = 18.1 deg
alpha_1 x 20 (more trans -> rot) | after   10 m: std(x) = 2.46 m (lateral),
std(distance to start) = 0.22 m, std(theta) = 27.0 deg
```



Observaciones:

- **Más α, más dispersión.** Con los α multiplicados por 0.1, 1, 4 y 10, la desviación típica final de la nube en y es de unos 0.5, 1.7, 3.2 y 4.5 m. Crece aproximadamente con $\sqrt{\alpha}$, porque los α multiplican varianzas, no desviaciones típicas. A partir de cierto punto crece menos, porque la nube se curva y deja de parecerse a una gaussiana.
- **Cada parámetro tiene un papel distinto**, y la fila inferior lo aísla:
  - α_0 (giro → error de giro) solo actúa cuando el robot gira. En este recorrido eso solo ocurre en la esquina ($\bar{\theta}_1 = -\pi/2$). Cada partícula sale de la esquina con un error de orientación fijo y luego avanza la misma distancia, así que las nubes son **arcos perfectos** centrados en la esquina. Pese a actuar una sola vez, produce la mayor parte de la dispersión en y (1.5 m).
  - α_1 (traslación → error de giro) añade un error de orientación en cada avance. La nube se ensancha perpendicularmente a la marcha desde el primer paso y se va curvando.

    - α_2 (traslación → error de traslación) solo alarga la nube en la dirección de avance, y muy poco (0.1 m), porque los errores de distancia no se amplifican al seguir avanzando.
    - α_3 (giro → error de traslación) solo actúa en la esquina y produce un pequeño error de distancia (0.14 m en x y nada en y).
- **Los errores de orientación dominan**, igual que en el modelo analítico: α_0 y α_1 abren la nube en abanico, mientras que α_2 y α_3 apenas la alargan.
- **Forma de la distribución (segunda figura).** Tras avanzar 10 m en línea recta, las partículas no forman una elipse, sino una **«banana» o media luna** que sigue el arco de radio d centrado en el punto de partida (línea discontinua). Con los α del enunciado el error de orientación es pequeño (6°) y la curvatura apenas se nota. Con $\alpha_1 \times 20$ (27° de error de orientación), la dispersión lateral es de 2.5 m y la radial de solo 0.2 m. Las elipses gaussianas con la misma media y covarianza (negro) encajan mal: su centro cae en una zona casi vacía, por debajo del arco, y dejan fuera las puntas de la media luna. Por eso los filtros de partículas usan la forma de muestreo.

Thinking about it (2)

Now you are an expert in the sample form of the odometry motion model! **Answer the following questions**:

- What is the effect of modifying the robot's intrinsic noise parameters $\boldsymbol{\alpha} = [\alpha_0, \ldots, \alpha_3]^T$ (`a` in the code)?

  Los α fijan la varianza de cada componente de la orden en función de cuánto se mueve el robot. Cuanto mayores son, más se dispersan las partículas: con α×0.1, ×1, ×4 y ×10, la desviación típica final de la nube en y es de unos 0.5, 1.7, 3.2 y 4.5 m, y crece aproximadamente con √α. Cada parámetro modela un efecto distinto:

  - ***α_0**: error de giro proporcional a lo que se gira. Solo actúa en la esquina, pero, como deja a cada partícula con una orientación distinta, produce arcos centrados en la esquina y casi toda la dispersión (1.5 m).*
  - ***α_1**: error de giro proporcional a lo que se avanza. Abre la nube perpendicularmente a la marcha desde el primer paso.*
  - ***α_2**: error de distancia proporcional a lo que se avanza. Solo alarga la nube en la dirección de avance (0.1 m).*
  - ***α_3**: error de distancia proporcional a lo que se gira. Solo se nota en la esquina y es muy pequeño.*

  Los parámetros que generan errores de orientación (α_0, α_1) son los que más ensanchan la nube, porque esos errores se amplifican con los avances siguientes. A diferencia de la Σ constante del modelo analítico, aquí la incertidumbre de un

paso es nula si el robot no se mueve y crece con la cantidad de movimiento, lo que es más realista.

- How many components does the odometry motion input $\mathbf{u}_t^{\text{odo}}$ contain?

  Tres: $\boldsymbol{u}_t^{odo} = [\bar{\theta}_1, \bar{d}, \bar{\theta}_2]^T$, un giro inicial para encarar el destino, un avance en línea recta y un giro final para alcanzar la orientación. Es otra forma de expresar la misma información que $[\Delta x, \Delta y, \Delta\theta]^T$: tres grados de libertad, que se calculan a partir de dos poses de odometría consecutivas con `generate_move()`. La ventaja es que cada componente corresponde a un movimiento físico que un robot no holónomo puede hacer. Así, al perturbarlas por separado, cada partícula sigue siendo un movimiento factible.

- After moving the robot a sufficient number of times, what shape does the distribution of samples take?

  *Toma forma de **«banana» o media luna**: un arco centrado aproximadamente en el punto de partida, con radio igual a la distancia recorrida. La dispersión a lo largo del arco (lateral) es mucho mayor que la radial: tras 10 m con $\alpha_1 \times 20$, 2.5 m frente a 0.2 m. Esto ocurre porque el error de distancia solo mueve las partículas hacia delante o hacia atrás, mientras que el error de orientación hace girar todo el avance alrededor del origen. Con α_0 solo, las nubes posteriores a la esquina son arcos perfectos. Esta distribución no es gaussiana: una elipse con su misma media y covarianza pone su centro en una zona casi vacía y no cubre las puntas. Es la limitación de la forma analítica, y la razón por la que los filtros de partículas usan la forma de muestreo, que puede representar distribuciones arbitrarias.*

AI Appendix

A.1 How I used AI

- **Model/tool:**

  Un asistente de programación basado en IA generativa (Devin con Claude Opus 5.5)

- **What I asked AI to help with (bullets):**
  - *Ayuda para completar `Robot`, `generate_move()` y `SampledRobot`, la parte opcional de odometría a partir de encoders y los experimentos con distintas `Q` y `a`.*
  - *Explicaciones de la propagación de la covarianza con jacobianos y de la forma de muestreo del modelo de odometría.*

- **Best prompt I used (paste):**

  «Ponte con el 3.3 y luego haz como con las otras y genera un doc explicándolas en profundidad, uno para cada»

- **What I kept vs. changed from the AI output:**

  He revisado el código contra las ecuaciones del notebook, he comparado las figuras con las de ejemplo y he comprobado con Monte Carlo que la covarianza propagada es correcta antes de quedarme con la solución.

A.2 AI review

Ask one generative AI to review your memo using this prompt:

```
Act as a technical reviewer of this robotics notebook. Read
my memo and return exactly 5 bullet points:

- two on theory (key ideas),
- two on practice (relevant programming concepts of python
used, not particular functions or variables),
- one on how to improve the code (most critical aspect:
limitations, parameter choices, runtime/robustness, possible
bugs).

Keep each bullet ≤15 words and make them specific to my work.
```

Paste its **5** items here:

AI used for the review: an AI coding assistant.

- **(Theory 1)** AI claim:

  Los jacobianos linealizan la composición y propagan la covarianza; Monte Carlo lo valida.

- **(Theory 2)** AI claim:

  La forma de muestreo captura la distribución en «banana» que una gaussiana no representa.

- **(Practice 1)** AI claim:

  Vectorización NumPy: todas las partículas se actualizan a la vez, sin bucles explícitos.

- **(Practice 2)** AI claim:

  Semillas fijas y funciones de simulación reutilizables hacen los experimentos reproducibles y comparables.

- **(Improvement)** AI claim:

  `generate_move()` *no normaliza ángulos; θ_1, θ_2 fuera de $(-\pi, \pi]$ inflan las varianzas.*

Finally, **do you agree with them?**, **would you have selected others?**

Estoy de acuerdo con los cinco puntos. Los teóricos resumen la idea central de la práctica: la forma analítica es cómoda (una media y una covarianza), pero solo aproxima, y la forma de muestreo representa mejor la distribución real cuando hay mucha incertidumbre angular. La mejora señalada es un fallo real: el ejemplo del enunciado devuelve $\bar{\theta}_1 = -3.93$ rad, que equivale a +2.36 rad. Como $\sigma_{\theta 1}^2 = \alpha_0 \bar{\theta}_1^2 + \alpha_1 d^2$, el robot «creería» haber girado 225° en vez de 135°, y el ruido sería mayor del necesario. No lo he cambiado porque la salida esperada del notebook es precisamente la versión sin normalizar, y en el recorrido de la demo no ocurre. Como alternativa habría señalado que `SampledRobot` *modifica in situ el array* `mean` *que recibe (la pose inicial del llamador cambia al moverse el robot). En la parte práctica habría destacado también la reutilización de las utilidades de la práctica 3.1 (* `tcomp`, `inv_tcomp` *) en lugar de reescribir la composición.*